ECM2414 - SWD CA – Design Rationale & Critical Report: Theo Swan & Henry Beedham

Code Architecture:

For our solution we designed and implemented 4 classes to handle the simulation: Card, CardDeck, Player and the main executable CardGame. Each card is represented by a Card object, in which we utilise encapsulation and the final keyword ensuring cards cannot be duplicated and remain constant. Each Deck is represented by a CardDeck object which enforces the required FIFO structure. Alongside this we have implemented synchronised to ensure resolution of race conditions and atomicity of individual card transactions. For the multi-threaded component of our program, the players, are represented by the Player class, handling the execution of their individual strategy, we have implemented atomicity through the implementation of locks on the decks to the 'left' and 'right' of the player. Finally, CardGame acts as our main executable, it handles the game's lifecycle, pack distribution, thread and file handling.
For the player class, we used an atomic integer to store the winner id because it can be accessed by all the player threads safely without the possibility of overwriting what is being actively written.

Our Decisions

We decided early on in our development process to utilise the Waterfall development cycle. It is composed of 5 stages: Requirements, Design, Implementation, Testing and Analysis. We thought this appropriate given the predefined specification alongside the risk iterative development would not produce an appropriate solution given our short timeframe. This cycle came at the particular benefit that we would not need to conduct major overhauls upon the analysis phase of our development following our testing.

Through the implementation of abstraction, we ensured a clear distinction between subproblems and that the responsibilities of each class are well defined. This aids us not only by reducing the complexity of the implementation through 4 less complex sub-components but also simplifies the testing process, as there are fewer complex subjects to design and implement testing for. Ultimately this will result in saved development time and stronger documentation, as it can be specialised.
Following the impact of Theo's circumstances and Henry's demanding schedule, we were faced with a difficult decision. We could either adapt our development approach or submit completely incomplete work. To meet the now static submission date we chose the former and adapted our approach. We instead implemented a Leapfrogging approach where both roles of passenger and driver still exist but simply not necessarily simultaneously. We recognise this was less than ideal, however there were no alternative options available to us, and it ensured equal contributions to the project, prevented any loss of learning opportunities and made the submission date possible to meet. We ensured to always review the others work, discussed any questions and suggestions before the development lifecycle further advanced. Furthermore, it also provided additional benefits providing a fresh eyes effect in the review process, allowing us to spot mistakes that would have otherwise been missed.

As this is a multi-threaded simulation we have considered thread safety throughout our classes. This has included the implementation of synchronised to ensure atomicity of any transactions between hands and decks. We recognised the risk of race conditions within our player class due to the shared nature of an individual deck between 2 players. We ensured atomicity of individual card transactions through the implementation of synchronised and atomic. However there remains the case that 2 players may discard and draw to and from the same deck simultaneously.

To handle the simultaneous processing of each player's strategy we have implemented the interface Runnable and overridden it's run() method. Through inheritance and polymorphism we have built the

individualised strategy into the run() method allowing thread creation, execution and termination to scale as required, given the variable players that can participate in a game. Furthermore, it reduces the complexity of the CardGame class given that we do not have to handle any threads lifecycle after creation.

Alternatives Considered:

We considered additional classes for classes for round robin and game state, we later realised this was not the appropriate approach and implemented these simulation aspects as methods within our main executable within CardGame.

We considered an alternative solution to our circumstances such as unbalancing the workload and implementing a rapid independent development process. However, this came with a host of major drawbacks. It would have been unfair to both of us, negatively impacted our grades, eliminated any learning opportunities, lead to a lower quality submission and risked the project missing the final deadline available to us.

Testing Strategy

We are using Junit 5.14.

We undertook a thorough testing strategy as by the end of implementation of a component it should be 100% functional, as our design process should have foreseen and prevented any major issues. However of course, no design and implementation is completely flawless and any errors that arose were often very minor and were quick to fix. Through taking this approach this allowed us to ensure we met far more requirements upon the first run of testing than would have happened in a Prototype heavy approach such as RAD, saving us significant amounts of time.

Through our unit testing, we conducted basic testing such as valid, invalid and edge case inputs alongside more advanced testing of specific behaviours, such as the FIFO behaviour of CardDeck, input pack validation, immediate win detection and correct formatting of output.txt files.

Limitations & Performance Issues

There is no resolution for 2 adjacent players taking a draw & discard action simultaneously on the same deck.
No other Known Limitations or Performance Issues.

Specification Fulfilment:

Our application satisfies the specification in the following ways:

- Use a software design and development method which has incorporated both formal and informal techniques appropriately: We used a formal waterfall development lifecycle and adapted to any changing circumstances, which occurred both in and outside the project. In addition, we utilised many different more informal techniques within the project itself, this included: threading, file input/output, enforcing preconditions, object-oriented programming and a thorough testing suite.

- Design and implement rigorous testing frameworks for software and be aware of the inherent limitations of tests developed: We developed a thorough testing suite which ensured that our program operated with the correct behaviour, enforced the necessary pre-conditions and was able to prevent unexpected behaviours through both valid and invalid inputs.

- Deploy advanced object-oriented language concepts and techniques: We have utilised key concepts and techniques including encapsulation, inheritance and polymorphism throughout our development to achieve the necessary requirements to ensure thread safety.

- Develop multi-program software systems: We have developed multiple class files for both our implementation and testing.

- Follow the phases of software development: We have followed a waterfall development cycle with clear phases such as design, implementation and testing.

- Recognise and evaluate different development practices and judge their appropriateness for a specific development problem: We believe our waterfall development lifecycle was the best one given our circumstances and formally defined requirements. By shifting to a Leap Frogging paired programming approach, it meant we gained more flexibility and were able to do significantly more work on the project, given our circumstances.

- Analyse and break down a problem into constituent parts. Compare and critically contrast different potential solutions to a problem: We did this through not only our multi program approach and our phased development cycle, but this also occurred before this within our 1st Paired Programming meeting and considered alternative ways to address certain subproblems such as the round robin algorithm.
- Follow the pair-programming development approach used widely in industry: We followed the paired programming approach giving us the benefits of equally spread workload, learning opportunities and a cleaner code base in general due to the constant rotation of active roles.

Development Record

Theo summarised CA spec independently - 10th Oct | Theo - Driver
1st CA meeting – 13th Oct 14:00 - 15:35 | Theo - Driver | Henry - Passenger

- Started writeup report - Created plan. GitHub Repository & File Hierarchy established.
- SHA: 8d0bb8b4a55d23d6d170142f35df4d128a7a391f

Henry formalised agreed class designs independently - 15th Oct | Henry - Driver

- Formalised all fields and methods for all classes. Created descriptions of fields & methods for easier implementation.
- SHA: c3e1bbc3e579b8d10acefe8d2a4042282c41b7f0

2nd CA meeting – 15th Oct 9:35PM – 22:08 | Henry - Driver, Theo – Passenger

- Henry explained what work he did. Theo asked questions and made suggestions.
- SHA: 2addc598c29875da0bdb1351595e2c5f2ed95ec4

Theo developed Card, set up testing – 15th Oct 22:10 - 03:40 | Theo – Driver

- Implemented Card Class. Set up Junit and Maven. Set up testing and tested Card.
- SHA: 2addc598c29875da0bdb1351595e2c5f2ed95ec4

Theo developed CardDeck – 16th Oct 18:14 – 20:49 & 23:00-2:40AM | Theo - Driver

- Implemented CardDeck Class. Tested CardDeck Class
- SHA: 77e366a6f7456094fa15263773e92d0c4ed9b413
- SHA: a725913467b430fdc5a15c4bd7afef852812d424

Henry developed Player and CardGame – 17th - 30th Oct | Henry – Driver

- Implemented Player and CardGame Classes
- SHA: d31653ad7c7b38bbaf3f1ebdd25bb7ac29f3bca5
- SHA: 79d1c5443f801d5833bf5d26194232418d500285

Theo reviewed code and overhauled the writeup – 1st Nov ~01:00-4:10AM | Theo – Passenger & Driver

- Reviewed Player and CardGame Classes. Raised race condition concern with Henry regarding 2 players drawing and discarding simultaneously. Fixed missed requirement to output winner to terminal – CardGame.java. Fixed formatting error in Player.java text output. Major Writeup Overhaul: Code architecture, Our Decisions, Alternatives considered.
- SHA: e9b204003193330790a81041db46034c808a16f0

Theo reviewed overhaul – 1st Nov 21:24 - 23:27 | Theo - Driver

- Reformatted Development Record + Added Commit ids + moved class designs to 'class designs.md' file. Started Testing strategy section
- SHA: 26b21e95d48d629d4c051a5957dcf07923b567ed

Henry wrote testing for player class - 3rd Nov 00:00-01:10

- Write testing for player class

Theo continued writeup & README.md, Henry made Player Tests – 2nd Nov 22:38 – 01:06 | Theo – Driver Henry - Driver

- Fixed bug in Player Card Discarding
- Wrote README.md, ran Maven and our Junit tests from scratch. (terminal)
- Henry made Player tests
- SHA: daa02896599e57ca8aa2d605ecb7518ea1025f29
- SHA: 99d8cf86a51369fe34ee5381ac5c1c45dd11fda7

3rd CA meeting – 3rd Nov 11:00 – 12:00 | Theo - Driver Henry - Driver

- Final work for coursework submission done.